

Eyefinity APIs – Overview



Change Log

Date:	Changes:	Version:
2022-09-23	Initial Release	1.0

Table of Contents

Change Log	i
Introduction	2
Getting Started.....	2
OAuth2.0 Authorization Code	2
Authorization Code Request	4
Access Token Exchange.....	5
Call The API.....	5
Refresh Expired Access Token	5
Webhooks	6
Timing	6
Encryption and Non-Repudiation	7
Subscription.....	7
Versioning	7
HTTP Status Codes	7
Errors.....	8

Introduction

Eyefinity APIs are designed to enable integration with our customers and their business partners. Our APIs provide smooth and secure communication between applications, which provides you with flexibility in accessing and sharing your data and often speeds integration with third-party services.

Eyefinity offers the following APIs:

- **Retail APIs** – access to Inventory Activity, Lab Orders, Office End-of-Day Activity and Office Invoice Activity generated in the Eyefinity system.
- **Claims APIs** – access to Claims Activity generated in the Eyefinity system.
- **Order Status APIs** – access to Order Status changes generated in the Eyefinity System and retrieval of Lab Order Status from external systems.
- **CRM and Patient Engagement** – access to Patient and Appointment data. CRM offers subscription API and uses webhook technology to push patient and/or appointment data when triggered by an event within the system, such as, a change in contact and insurance information or creating/cancelling an appointment. With Patient Engagement, you can use the APIs to pull patient and appointment data.
- **Bulk Data API** – take a snapshot of your Eyefinity system data, which can be consumed by a third-party service.

Getting Started

To get started with data sharing, you need to set up a secure way to access your data.

OAuth2.0 Authorization Code

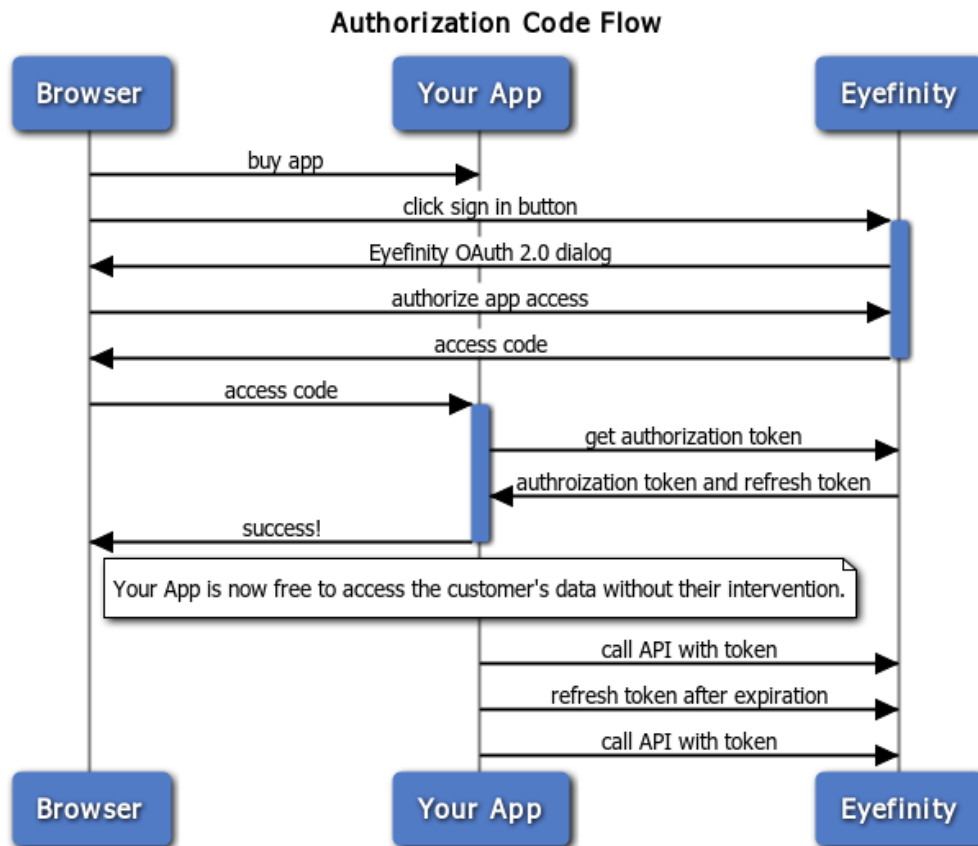
Eyefinity APIs require integrated applications to use the OAuth2.0 Authorization Code Flow.

In this flow, your application:

- Obtains permission (an authorization code) from your customer via the Eyefinity's OAuth 2.0 (EO2) gateway server,
- Obtains an authorization token from EO2 so your application can call the API on the customer's behalf, and
- Refreshes the authorization token with EO2 when the original authorization token expires.

Note: In certain scenarios, Eyefinity also supports the Client Credential Flow. Details are available upon request.

The graphic below illustrates the Authorization Code Flow.



To support the OAuth 2.0 Authorization Code Flow, you need to securely store five pieces of information:

- **Client ID** – an identifier assigned to your application by Eyefinity. The Client ID is used in all OAuth calls and all calls you make to our APIs.
- **Authorization Token** – a security token (associated with each of your customers) that provides access to their specific data. It must be supplied as an Authorization header in all calls made to our API. This token expires after a short time frame, so your application will need to "Refresh" the token (or, otherwise, have the user log in and authorize data access every time).
- **Refresh Token** – a security token (associated with each of your customers) that allows your application server to re-authenticate and obtain a new authorization token without user intervention.
- **Token Expiration** – the date-time your customer's authorization token expires. This helps your application determine when to refresh the authorization token.
- **API Gateway Domain** – the Internet domain name of Eyefinity's OAuth gateway. Use api-sandbox.eyefinity.com for developer testing and api.eyefinity.com for production.

Authorization Code Request

Parameters

Name	Description
client_id	The value provided when you registered your application with Eyefinity.
response_type	"code" - indicating an authorization code will be returned to the application after the user approves the authorization request.
scope	Scopes define the extent of data that your application is requesting access to, specified as a list of space-delimited strings. For example: for Retail Integrations, use "auth_epm ef.skynet_poslog" as the scope. auth_epm Authenticate as an Eyefinity user ef.skynet_poslog download POS Log
redirect_uri	The location the user is returned to after they approve access for your app. You must supply this URI to Eyefinity before you can access the APIs directly. If this does not match the value on our gateway, your request will fail.
state	A unique value generated by your application in order to prevent cross-site request forgery (CSRF) attacks. The value should be a random unique string for each particular request, unguessable and kept secret in the client (perhaps in a server-side session).

Sample Response:

If everything goes well during the authorization code request, then the OAuth server redirects to your application's call back URL with the following query string parameters:

- **code**: contains the authorization code
- **state**: contains the original session id sent in with the request to prevent cross-site forgery requests

If there is an error or access is denied, the OAuth server will respond with the following query string parameters

- **error**: contains error code, i.e., access_denied, invalid_request, unauthorized_client, invalid_scope, server_error, temporarily_unavailable
- **state**: contains the original session id sent in with the request to prevent cross-site forgery requests

Success:

```
https://yourapplication.example.com/OauthCallback?code=38593485789789338995390892024890859389&state=C9E95DDCA390941664DEAE089AF7361
```

Failure:

```
https://yourapplication.example.com/OauthCallback?error=access_denied&state=C9E95DDCA390941664DEAE089AF7361
```

Access Token Exchange

Parameters

Name	Description
client_id	The value provided to you when you registered your application with Eyefinity.
secret	The confidential secret provided to you when you registered your application with Eyefinity.
redirect_uri	The location the user should be returned to after they approve access for your app. You must supply this URI to Eyefinity before you can access the API directly.
code	The authorization code that was returned to your application by a previous Authorization Code Request.
grant_type	"authorization_code" - indicating that you're exchanging an authorization code for an access token.

Call The API

Parameters

Name	Description
access_token	The token, returned by a previous Access Token Exchange or Refresh Expired Access Token call, that can be used to authorize API requests.

Refresh Expired Access Token

Parameters

Name	Description
client_id	The value provided to you when you registered your application with Eyefinity.
secret	The confidential secret provided to you when you registered your application with Eyefinity.
refresh_token	A token, returned by a previous Access Token Exchange, that your application uses to acquire a new access token (after the current one expires) without requiring your user to re-authenticate and authorize you to access their data.
grant_type	"refresh_token" - indicating that you are refreshing an expired access token.

Webhooks

Eyefinity APIs offers “push” notifications of events from the system. These notifications are implemented with webhooks technology, eliminating the need for API polling by sending near real-time data to user specified endpoints in an easy, less resource intensive way (see Figure 1 for visual representation). Once you complete a onetime subscription, webhook technology supports:

- Client managed subscriptions to available events
- System-generated events sent to subscribers at their registered API endpoints
- Transport security through encryption and non-repudiation

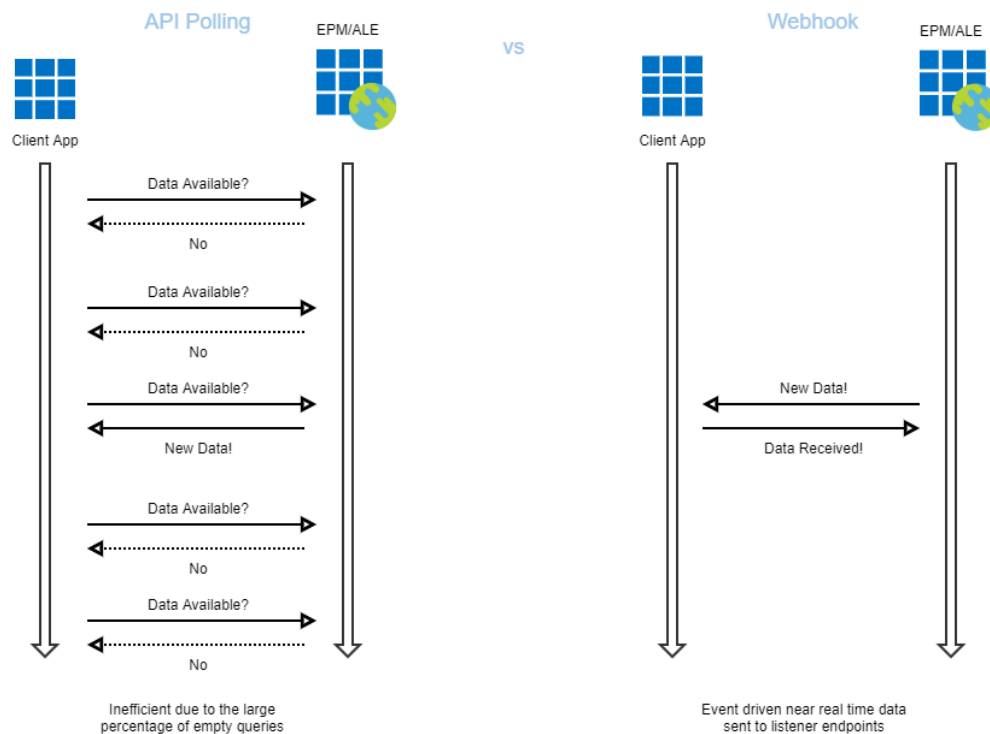


Figure 1

Timing

Webhook calls are generated shortly after the triggering event. The data is collected and formatted and sent via a webhook call to registered subscription points. Timing is in the order of seconds.

Encryption and Non-Repudiation

Webhooks are implemented with outward API calls that use TLS 1.2 or better for encryption-in-transit security.

When a subscriber registers for events of a given type, they will receive the public key of a private/public key pair generated by Eyefinity. This public key needs to be kept by the subscriber for the life of the subscription. Each subscription has its own private/public key pair rather than each subscriber. This distinction is important.

Each webhook transaction includes a JWT (JSON Web Token) that includes a signature segment. This segment is signed with the private key for that subscription. The subscriber should always verify this signature using the provided public key issued at the time of the subscription.

Subscription

Webhook subscriptions are currently administrated by Eyefinity support, you will only need to complete the process once per webhook type. To subscribe, you just need to supply the webhook type and the public endpoint URL for where you'd like the event payloads to be sent. As the Eyefinity platform evolves, the plan is to transition to a self-serve model via API or UI where users will be free to manage their own subscriptions once account registration is complete.

Versioning

Eyefinity commits to following the Semantic Versioning 2.0 specification, following the MAJOR.MINOR.PATCH convention.

Specifically:

- MAJOR version is incremented when we make incompatible API changes.
- MINOR version is incremented when we add functionality in a backwards compatible manner.
- PATCH version is incremented we you make backwards compatible bug fixes.

HTTP Status Codes

The following common HTTP Status Codes are used to describe the result of the REST request.

- **200 OK** - Response to a successful GET, PUT, PATCH or DELETE. Can also be used for a POST that doesn't result in a creation.
- **400 Bad Request** - The request is malformed, such as if the body does not parse.
- **401 Unauthorized** - When no or invalid authentication details are provided.
- **403 Forbidden** - When authentication succeeded but the authenticated user has not authorized your application to access the resource, or the user doesn't have access to the resource at all.
- **404 Not Found** - When a non-existent resource is requested.

- **410 Gone** - If the requested version is not supported (see "Versioning", above).
- **415 Unsupported Media Type** - If an incorrect content type was provided as part of the request.
- **500 Internal Server Error** - When there was an error in the API.

Errors

Most API-related errors will generate a JSON response that contains information about the error. It will look like the following:

```
code: integer
error: string
description: string
details: []
  code: string
  description: string
```

Below is a sample list of error codes returned by the API and what they mean.

1:	SystemException
2:	ApplicationError
3:	InvalidRequest
4:	InvalidFormat
5:	NumericOverflow
6:	ProviderNotFound
7:	ServiceNonFunctional
8:	AppointmentNotFound
9:	OfficeNotFound
10:	ResourceNotFound
11:	InvalidAPIUser
12:	OrderNotFound
13:	PatientNotFound
14:	CompanyNotFound
15:	RecallTypeNotFound
16:	AuthenticationError
40407:	POSLogNotFound